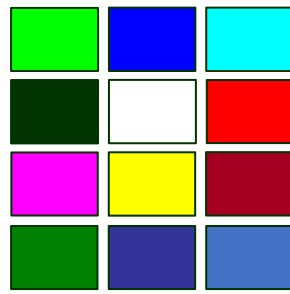


Topic 2

Combinational Components and their modelling using HDL

Gates and Modeling

Outline

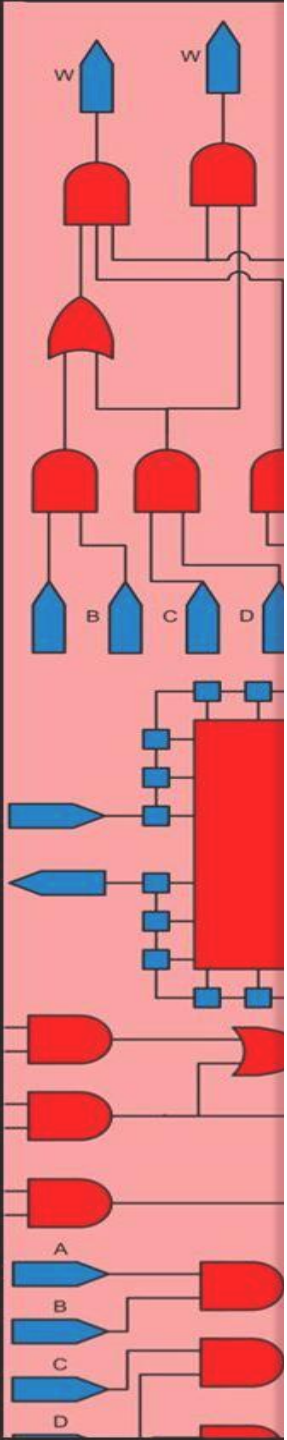


3-1 NMOS and PMOS transistors

3-2 Building Gates

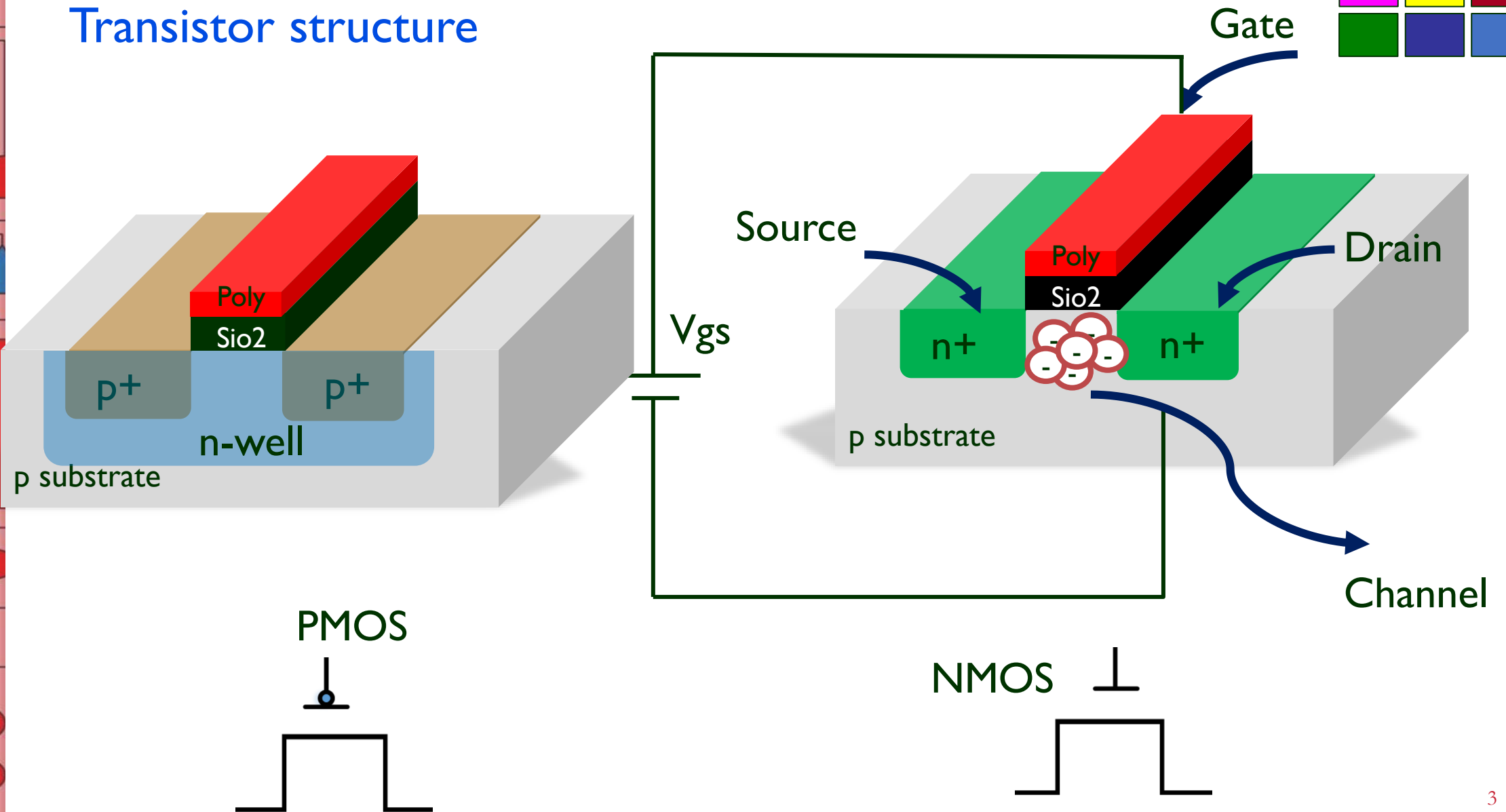
3-3 Complex Structures

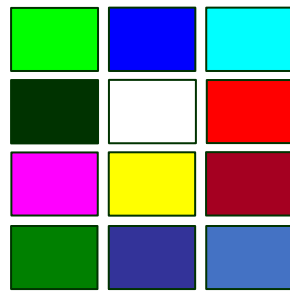
3-4 Overflow Detector Example



NMOS and PMOS Transistors

Transistor structure





NMOS and PMOS Transistors

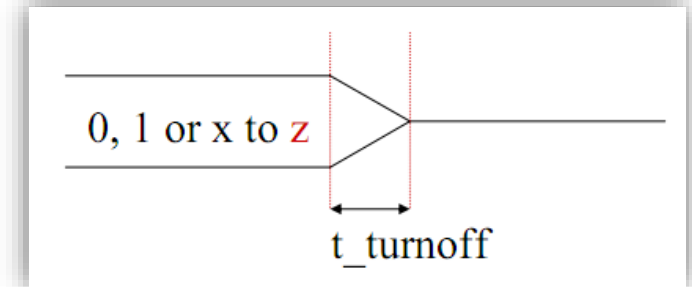
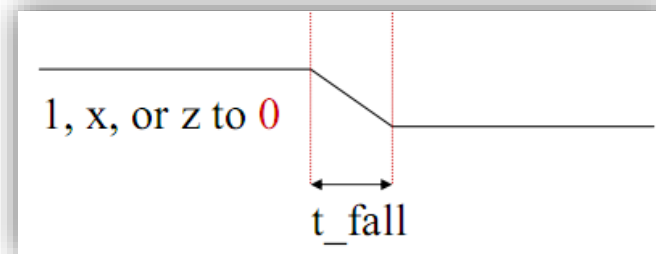
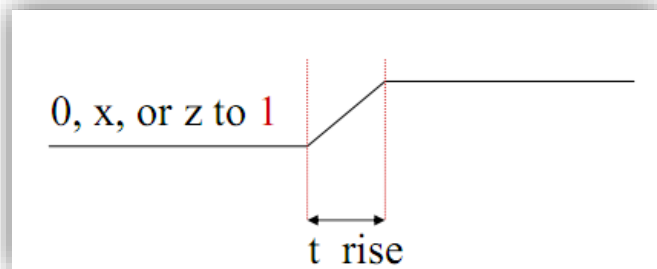
Transistor model

- Specify no delay

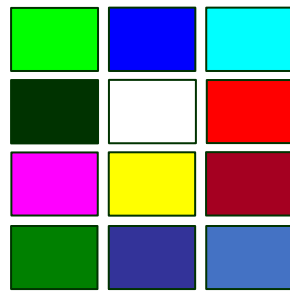
```
nmos [instance_name] (d, s, g)
```

- Specify rise, fall and turn off times

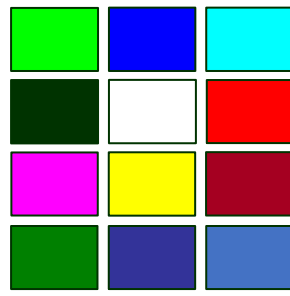
```
nmos # (ToI,To0,ToZ) [instance_name] (d, s, g)
```



4- Value Logic Model



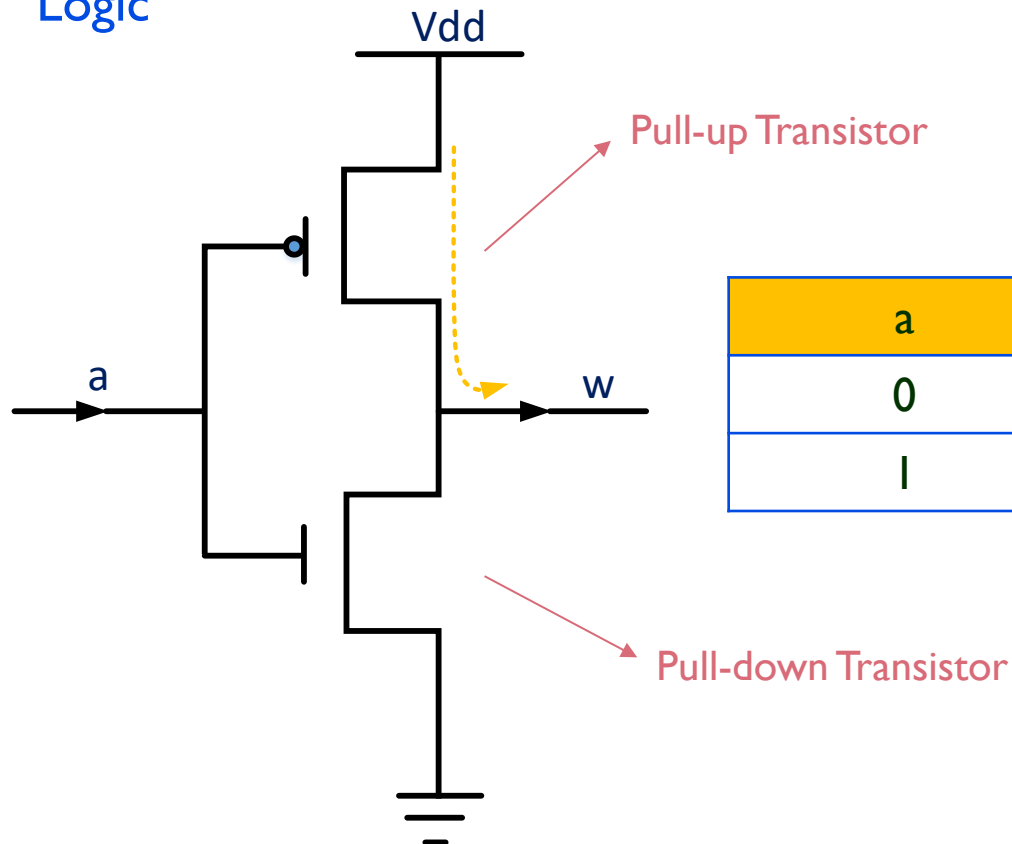
Value Level	HW Condition
0	Logic zero, False
1	Logic one, True
X	Unknown
Z	High impedance



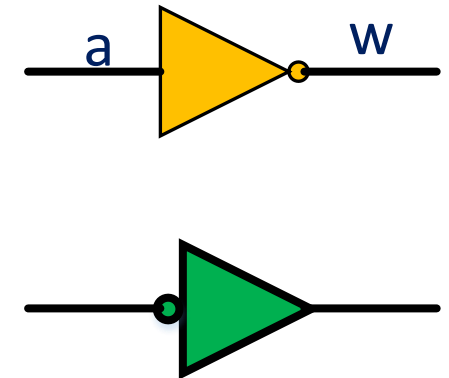
Building Gates

Inverter

- Input “0” will make the **PMOS** transistor **conduct** and the **NMOS** one **not conduct**, resulting in a “1” output
- Input “1” will have the output pulled down to “0” through the NMOS transistor
- Logic



a	w
0	1
1	0

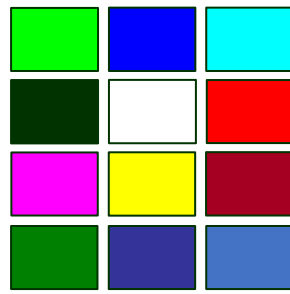


$$w = \bar{a} = \sim a$$

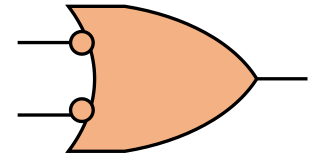
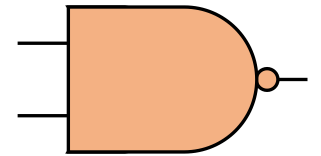
Building Gates

NAND gate

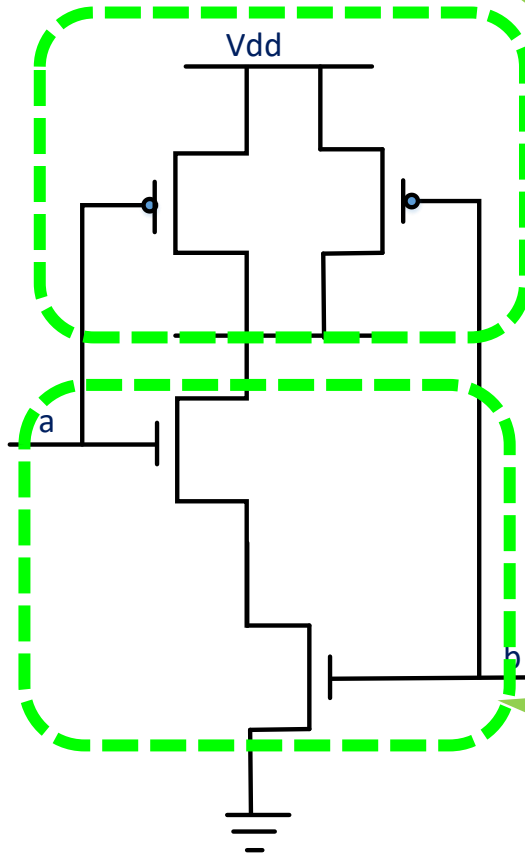
Structurally
Complementary



$$w = \overline{a \cdot b}$$



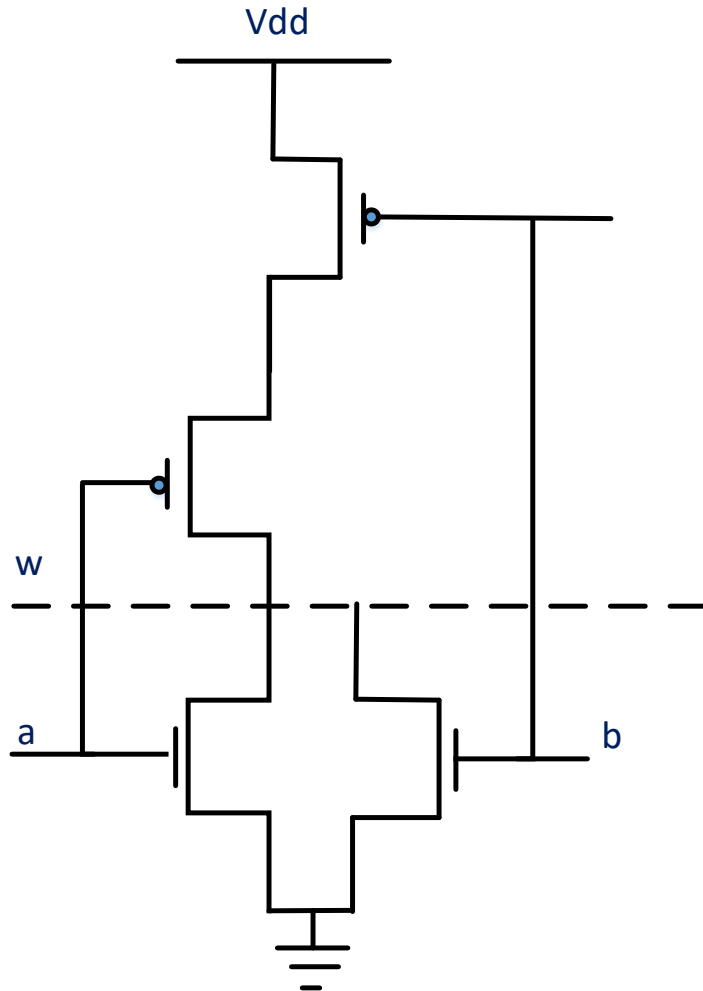
$$w = \overline{a} + \overline{b}$$



a	b	w
0	0	1
0	1	1
1	0	1
1	1	0

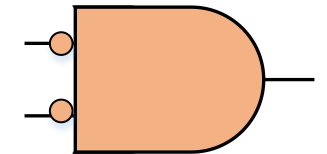
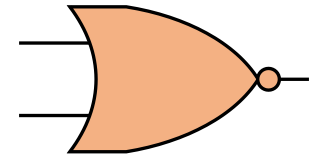
Building Gates

NOR gate

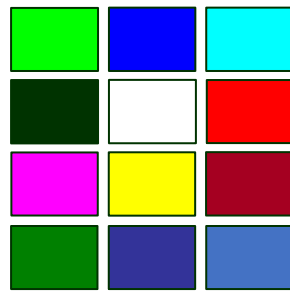


a	b	w
0	0	1
0	1	0
1	0	0
1	1	0

$$w = \overline{a + b}$$

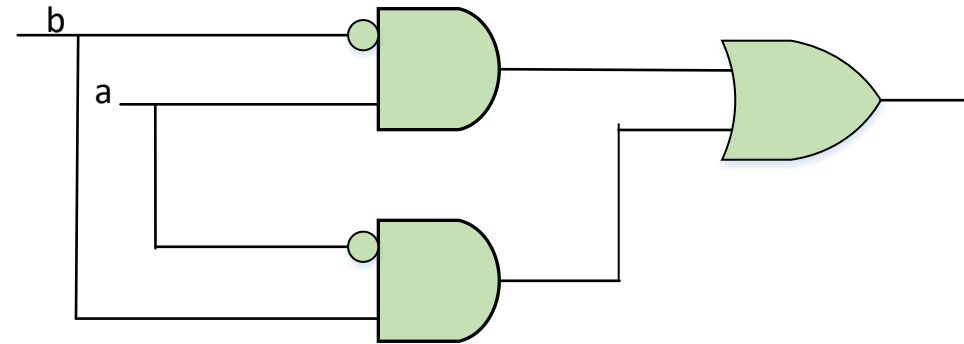
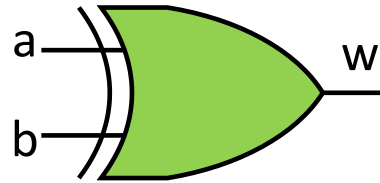


$$w = \overline{a} \cdot \overline{b}$$

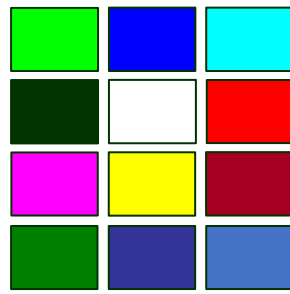


Complex Gates

XOR gate

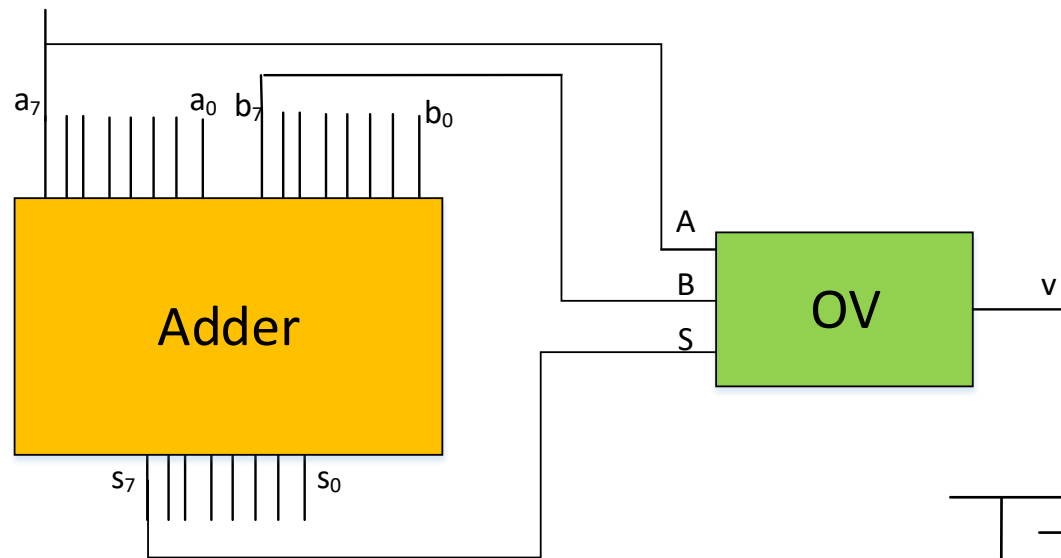
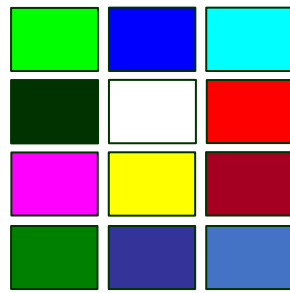


$$y = \bar{a}.b + \bar{b}.a$$

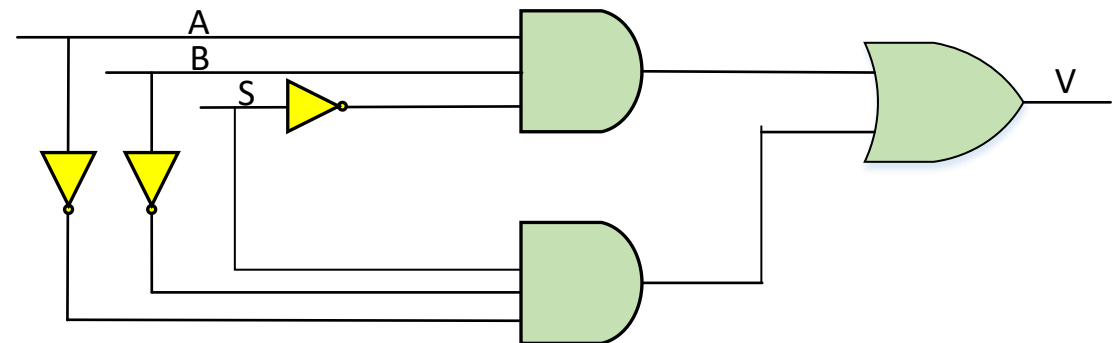


Example

Overflow Detector – Building the first circuit using gates

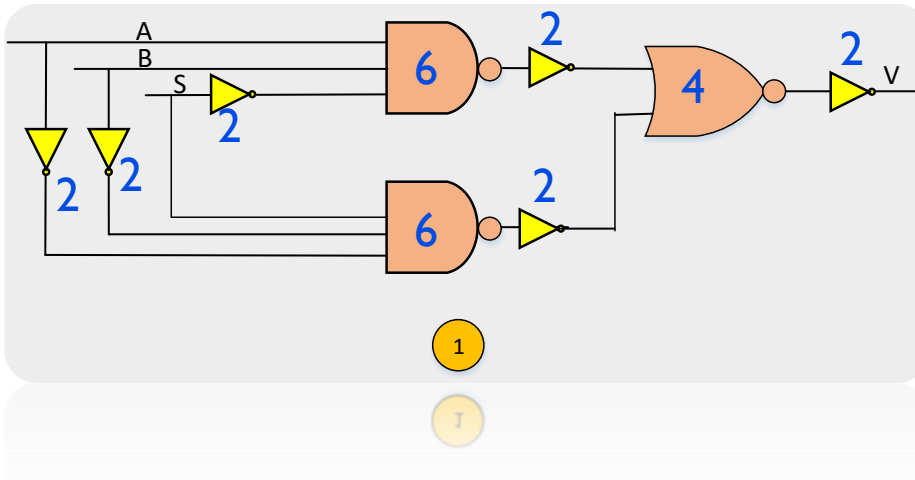
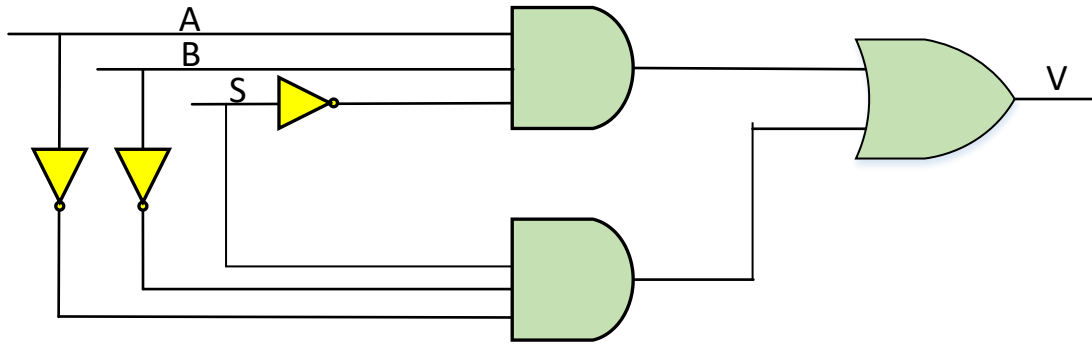
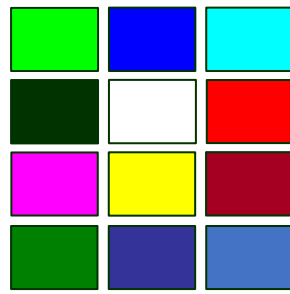


$$v = \overline{A} \cdot \overline{B} \cdot S + \overline{S} \cdot A \cdot B$$

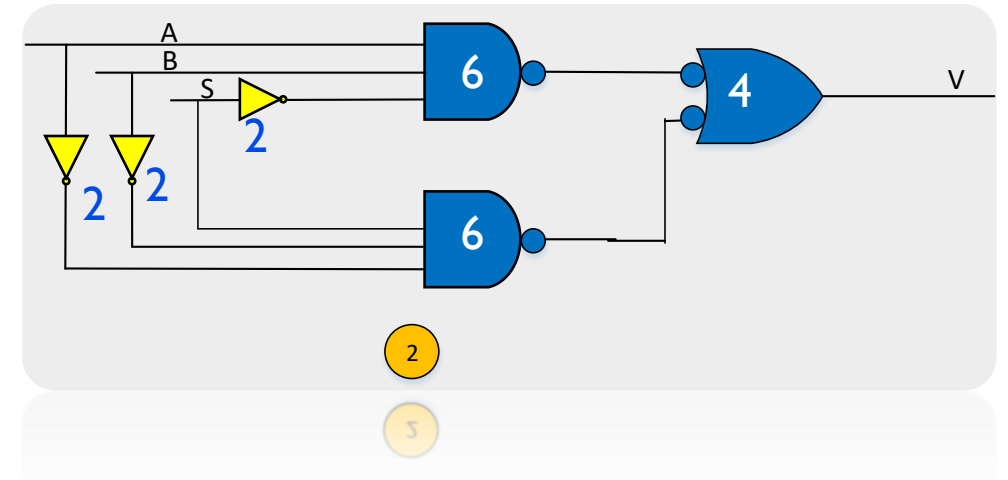


Example

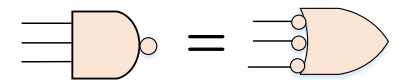
Overflow Detector

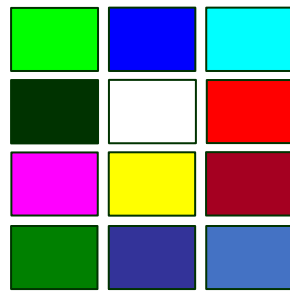


#transistor = 28



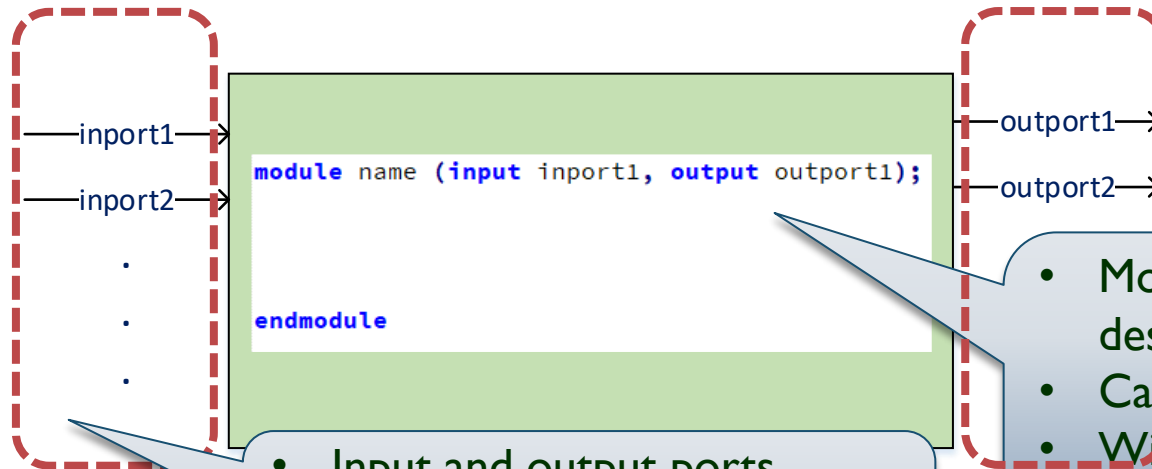
#transistor = 22





Building Gates

Inverter System Verilog description



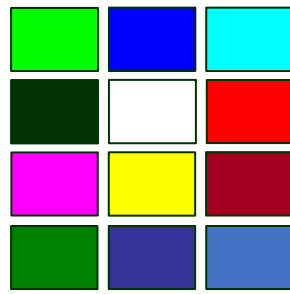
- Input and output ports connect the design to the outside environment

- Module is the building block of any design
- Can be instantiated in other modules
- Will be written inside a boundary (module and endmodule)

not **#delay** [instance_name] (out, in)

Example

Overflow detector SystemVerilog description



```
`timescale 1ns/1ns
```

```
module ov (
```

```
    input logic a,
```

```
    input logic b,
```

```
    input logic s,
```

```
    output logic v
```

```
);
```

```
    logic ai, bi, si, j, k;
```

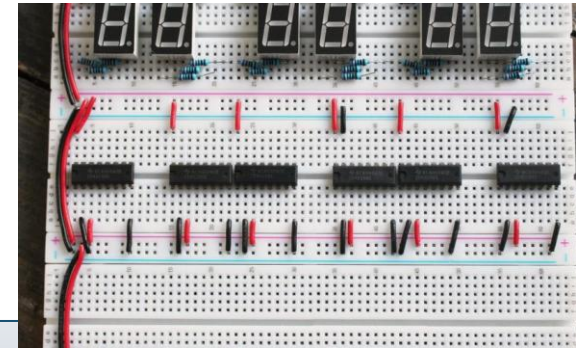
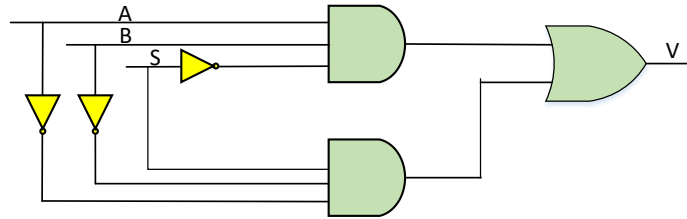
```
    or #10 R1(v, j, k);
```

```
    not #5 N1(ai, a), N2(bi, b), N3(si, s);
```

```
    and #15 A1(j, ai, bi, s);
```

```
    and #15 A2(k, a, b, si);
```

```
endmodule
```

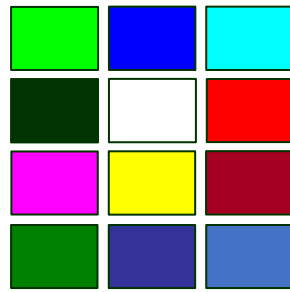


The order doesn't matter
Concurrent Structures

Can be replaced by:
`assign #40 v=(~a & ~b & s) | (a&b&~s);`

Example

Overflow detector SystemVerilog description



```
`timescale 1ns/1ns
module ov_tb;
  logic aa = 0, bb = 0, ss = 0;
  logic vv;
```

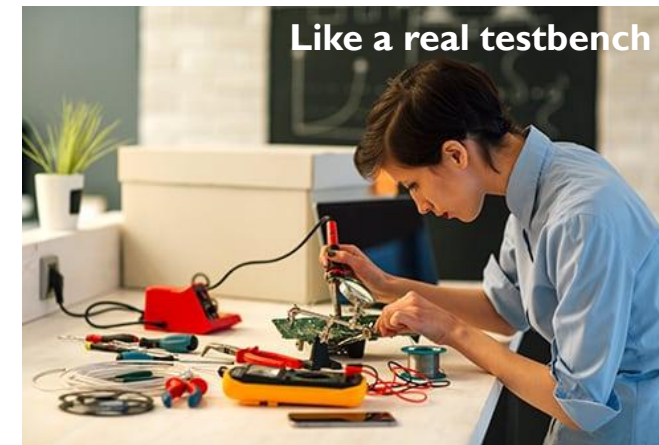
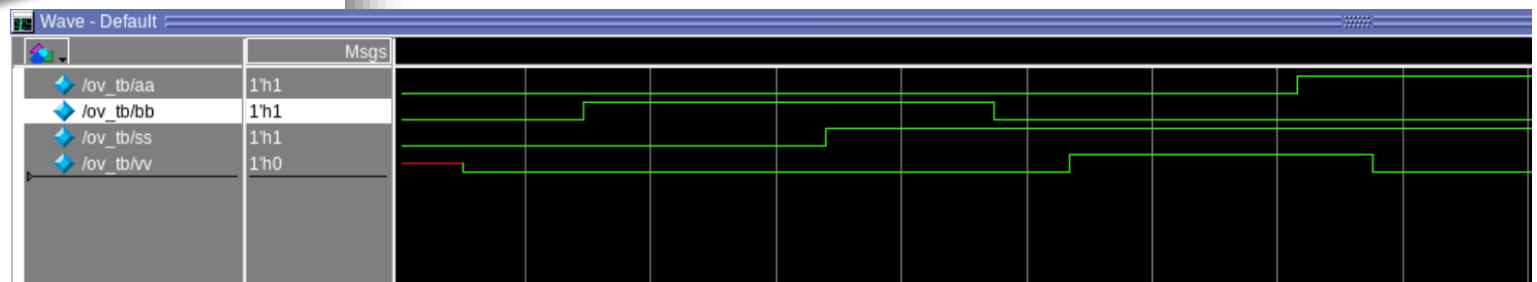
```
  ov uut (
    .a(aa),
    .b(bb),
    .s(ss),
    .v(vv)
```

Instantiation of the circuit

The order matters
Procedural Structures

```
);
initial begin
  #73  bb = 1;
  #97  ss = 1;
  #67  bb = 0;
  #121 aa = 1;
  #121 bb = 1;
  #137 ss = 0;
  #121 $stop;
end
```

```
endmodule
```



Conclusion

We have covered these topics:

- Transistor model
- How to build primitive gates
- How to describe a circuit in SystemVerilog
- How to write a testbench in SystemVerilog to test the circuit

